Heaven's Light Is Our Guide

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering

A report on "Project Title"

Course Code: CSE 3100

Course Title: Web Based Application Project

Authors

(Nazifa Anjum)

(2203147)

(C)

Supervisor

(Md.Sozib Hossain)

(Lecturer)

Dept. of CSE, RUET

Date: 13/07/2026

Rajshahi University of Engineering & Technology
Department of Computer Science & Engineering

Certification

This is to certify that the project titled: "***Learnify-Learning Management System***" submitted by ***[Nazifa Anjum]***, Roll Number(s) ***[2203147]***, for the fulfillment of the requirements of the course CSE 3100 (Web Based Application Project), has been examined and approved by the undersigned.

The project has been carried out under my supervision, and it is recommended for submission and evaluation.

Supervisor:
Name: Md.Sozib Hossain
Designation: Lecturer
Department: Computer Science and Engineering
Institution: Rajshahi University of Engineering and Technology
Date: 13.07.2026

Signature: _____

Abstract

The rapid growth of online education has increased the demand for effective Learning Management Systems (LMS) that provide engaging and accessible learning experiences. However, many existing platforms either focus mainly on course delivery or require multiple external tools to support features such as secure authentication, online payments, and student engagement. This project presents Learnify, a modern web-based Learning Management System designed to provide an interactive platform for both educators and students.The system was developed using React.js and Tailwind CSS for the frontend, Node.js and Express.js for the backend, and MongoDB for database management. Clerk was integrated for secure user authentication, Stripe for online course payments, and the YouTube API for video-based learning. The platform enables educators to create and manage courses, upload lectures and learning resources, while students can enroll in courses, watch video lectures, download resources, track learning progress, rate courses, and participate in a gamification system featuring XP, levels, badges, and leaderboards.The completed system was tested to ensure that all major functionalities operated correctly and provided a responsive user experience across different devices. Performance evaluation showed efficient API response times, smooth course navigation, and reliable data management. Compared with traditional LMS platforms, Learnify offers an engaging learning environment by integrating course management, secure payment processing, progress tracking, and gamification within a single application. The project demonstrates that a modern web-based LMS can improve the online learning experience by increasing student motivation, simplifying course management, and providing a scalable foundation for future enhancements such as discussion forums and advanced learning analytics. Learnify serves as a practical and cost-effective solution for educational institutions and online learning providers seeking an efficient digital learning platform.

List of Figures

List of Tables

Chapter-1
Introduction

1.1 Background of the Project

The rapid growth of online education has increased the demand for efficient and user friendly Learning Management Systems (LMS). Many existing platforms offer online learning services, but they may be expensive, difficult to manage, or lack features such as progress tracking, gamification, live classes, and downloadable learning resources.

To address these issues, this project, Learnify – Learning Management System, was developed. It provides a single platform where educators can create and manage courses, while students can purchase courses, watch video lectures, download study materials, track their learning progress, and earn points through gamification. The system aims to make online learning more accessible and engaging.

1.2 Objectives

To develop a secure and user-friendly Learning Management System that supports online teaching and learning.

Specific Objectives

- To allow educators to create, edit and publish courses.
- To enable students to browse and enroll in courses through secure online payment.
- To provide video lectures with downloadable learning resources.
- To track student progress and completed lectures.
- To increase student engagement through gamification features such as XP, levels, badges and streaks.
- To provide a responsive and easy to use interface for both educators and students.
- To enable students to rate the courses

1.3 Scope of the Project

The Learnify system focuses on providing the essential features required for an online learning platform.

Included Features:

- User authentication and authorization.
- Course creation and management.
- Chapter and lecture management.
- Video lecture support using YouTube.
- Downloadable lecture resources (PDFs,docs etc.).
- Secure course enrollment using Stripe payment.
- Student progress tracking.
- Course rating.
- Gamification features including XP, levels, badges, streaks, and leaderboard.
- Responsive web interface.

Limitations:

- The system currently supports YouTube video links instead of direct video hosting.
- There is no built-in quiz or examination module.
- Certificates are not generated automatically after course completion.

1.4 Report Organization

This report is organized into nine chapters, each covering a different aspect of the project.

- **Chapter 1: Introduction** presents the background of the project, objectives, scope, and the overall organization of the report.
- **Chapter 2: Background Study & Related Work** discusses the theoretical concepts, reviews existing Learning Management Systems, compares their features, identifies their limitations, and explains the need for the proposed system.
- **Chapter 3: System Analysis & Requirements** identifies the stakeholders, functional and non-functional requirements, and evaluates the technical, economic, social, and ethical feasibility of the project.
- **Chapter 4: System Design** describes the overall system architecture, database design, Entity Relationship (ER) diagram, and user interface design.
- **Chapter 5: Implementation** explains the development environment, technologies used, system modules, code structure, and important implementation details.

- **Chapter 6: Testing & Validation** presents the testing strategy, system testing results, and user acceptance testing to verify the correctness and usability of the system.
- **Chapter 7: Project Management & Teamwork** describes the project planning process, work breakdown structure, development timeline, budget estimation, risk management, and project execution.
- **Chapter 8: Results & Discussion** presents the system outputs, performance evaluation, screenshots, and comparison of the proposed system with existing Learning Management Systems.
- **Chapter 9: Conclusion & Future Work** summarizes the achievements of the project, discusses its limitations, and suggests possible future enhancements to improve the system.

Chapter-2
Background Study & Related Work

2.1 Theoretical Foundation

A Learning Management System (LMS) is a platform that helps educators create and manage online courses while allowing students to access learning materials anytime. This project is built using the MERN Stack (MongoDB, Express.js, React.js, and Node.js), with Clerk for authentication, Stripe for secure payments, and REST APIs for communication between the frontend and backend.

2.2 Overview of Existing Systems

1. Udemy
Introduction: An online course marketplace offering skill-based learning.
Features: Video courses, ratings, certificates, lifetime access.
Limitations: Limited structured learning paths and weak engagement features.

2. Coursera
Introduction: Offers university-level online courses and certifications.
Features: Professional certificates, quizzes, graded assignments.
Limitations: High cost and complex interface for beginners.

3. Moodle
Introduction: Open-source LMS widely used in educational institutions.
Features: Course management, quizzes, assignments.
Limitations: Outdated UI and complex setup.

4. Google Classroom
Introduction: Classroom management tool by Google.
Features: Assignment sharing, grading, announcements.
Limitations: Limited advanced learning and analytics features.

5. Khan Academy

Introduction: Free learning platform for school-level education.
Features: Video lessons, practice exercises.
Limitations: Limited course variety and no educator monetization.

Comparative Analysis

System	Key Features	Strengths	Weaknesses
Udemy	Course marketplace, video lectures, lifetime access, secure payment	Large course collection, affordable courses, easy course publishing	Limited interaction between students and instructors, no built-in gamification
Coursera	University courses, certificates, quizzes, assignments	High-quality academic content, recognized certificates	Many courses require payment, limited customization for individual instructors
Moodle	Course management, quizzes, assignments, grading	Open-source, highly customizable, widely used in institutions	Complex setup, many advanced features require plugins
Google Classroom	Assignment management, announcements, Google Workspace integration	Easy to use, free, excellent integration with Google tools	No payment system, limited course management and gamification features
Khan Academy	Free educational videos, exercises, progress tracking	Completely free, high-quality educational content, self-paced learning	Does not support instructor-created courses, no payment or live class features

Learnify (Proposed System)	Course creation, secure payment, video lectures, downloadable resources, live classes, progress tracking, gamification	Combines multiple learning features in one platform, modern UI, secure authentication, student engagement through gamification	Currently lacks quizzes, certificates, and discussion forums

Comparative Analysis of Existing LMS Pla *1*

2.3 Identified Gaps

The analysis of existing Learning Management Systems revealed several limitations:

- Most platforms do not provide built-in gamification features such as XP, badges, and learning streaks.
- Free platforms like Google Classroom and Khan Academy do not include secure online payment.
- Existing systems often lack a combination of course management, payments, downloadable resources, and progress tracking in one platform.
- Some platforms are expensive or have limited customization for educators.

2.4 Justification of Proposed System

The proposed system, Learnify, addresses these limitations by integrating multiple learning features into a single platform. It enables educators to create and manage courses while allowing students to securely purchase courses, watch video lectures, download learning resources, and monitor their progress. Additionally, the gamification system (XP, levels, badges, and streaks) improves student motivation and engagement, making the learning experience more interactive and effective.

Chapter-3
System Analysis & Requirements

3.1 Stakeholder Analysis

Stakeholder	Needs
Students	Browse courses, purchase courses, watch lectures, download resources, track progress and rate courses.
Educators	Create and manage courses, upload lectures and resources and monitor enrolled students

Stakeholder Analysis 1

3.2 Functional Requirements

The system should provide the following functionalities:

- User registration and login.
- Course creation and management.
- Chapter and lecture management.
- Video lecture streaming.
- Downloadable learning resources.
- Secure course enrollment and payment.
- Progress tracking and lecture completion.
- Course rating.
- Gamification (XP, levels, badges, streaks, leaderboard).

3.3 Non-Functional Requirements

Requirement	Description
Performance:	Fast page loading and quick API response
Security:	Secure authentication, authorization, and online payments.
Usability:	Simple, user-friendly, and responsive interface.
Reliability:	Stable system with accurate data management.

Scalability:	Able to support more users and courses in the future.
Maintainability:	Easy to update and extend with new features.

3.4 Feasibility Study (Engineering & Society)

Technical Feasibility

The system is technically feasible because it is developed using the MERN Stack, Clerk Authentication, and Stripe Payment Gateway, which are reliable and widely used technologies.

Economic Feasibility

The development cost is low since most tools and frameworks used are open-source or provide free tiers. The benefits of providing an online learning platform outweigh the development cost.

Societal Impact and Usefulness

The system supports online education by allowing students to learn anytime and anywhere. It also enables educators to reach more learners through digital courses and live classes.

Ethical Considerations and Accessibility

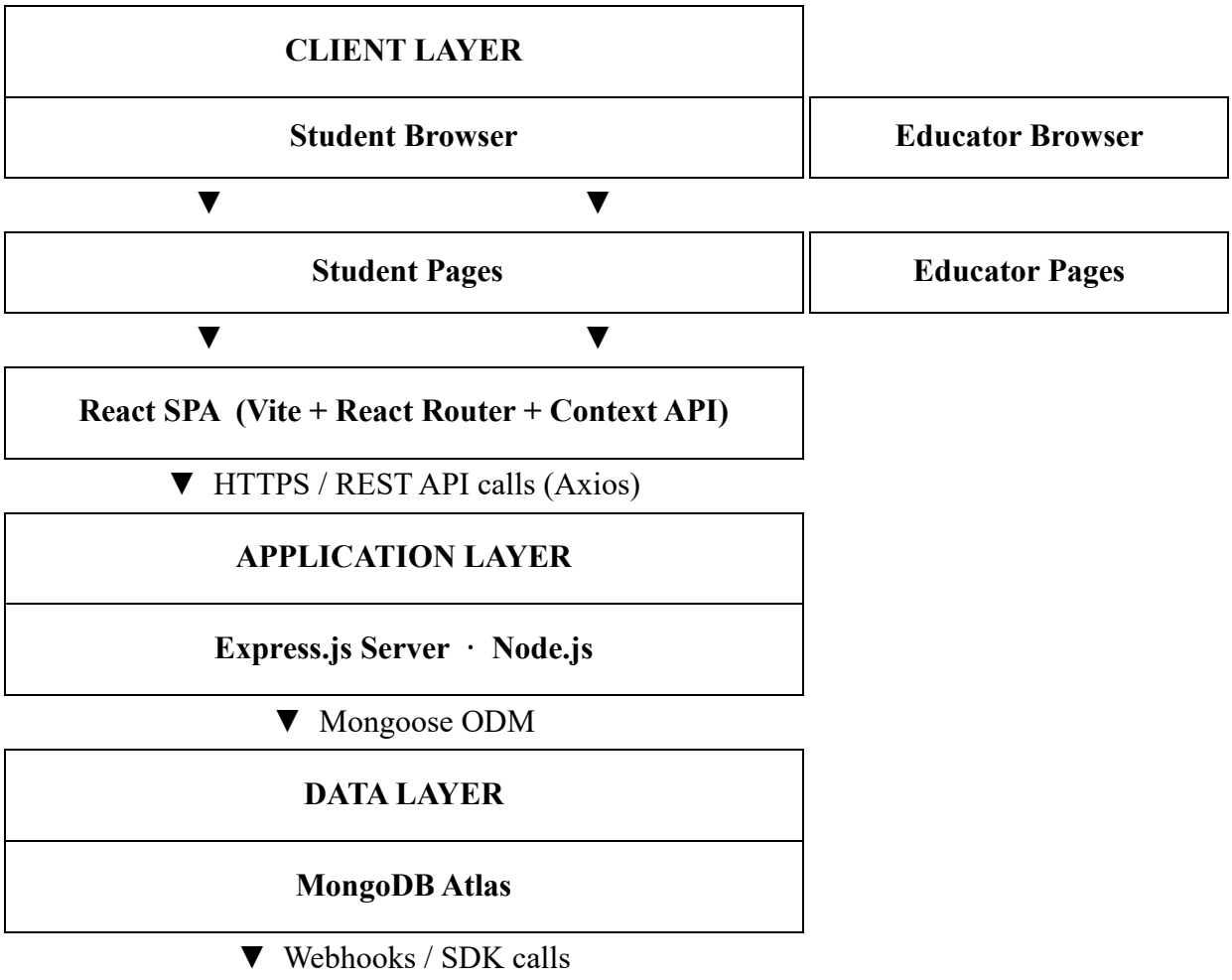
The system protects user data through secure authentication and payment processing. It promotes equal access to education through a responsive web interface that can be used on different devices and browsers.

Chapter-4
System Design

4.1 Architecture Design

The system is a web-based Learning Management System (LMS) built with the MERN stack (MongoDB, Express, React, Node.js). It follows a three-tier architecture: a React frontend that users interact with in the browser, an Express backend that handles all business logic and API requests, and a MongoDB database that stores all data.

Architecture Diagram



EXTERNAL SERVICES	
Clerk (Authentication)	**Stripe (Payments)**

System Architecture 1

How the Layers Work

Client Layer (Frontend)

The frontend is a React application that runs in the user's browser. It has two separate portals — one for students and one for educators — each with their own set of pages. The application uses React Router to move between pages without reloading, and Context API to share data like the logged-in user and course list across all pages.

- Student pages: Dashboard, Course List, Course Details, My Enrollments, Video Player
- Educator pages: Dashboard, Add Course, My Courses, Course Management, Student Enrollments

Application Layer (Backend)

The backend is a Node.js server running the Express framework. It receives requests from the frontend, applies business rules, and returns responses. Every protected route checks the user's identity using a JWT token issued by Clerk before processing the request.

Data Layer (Database)

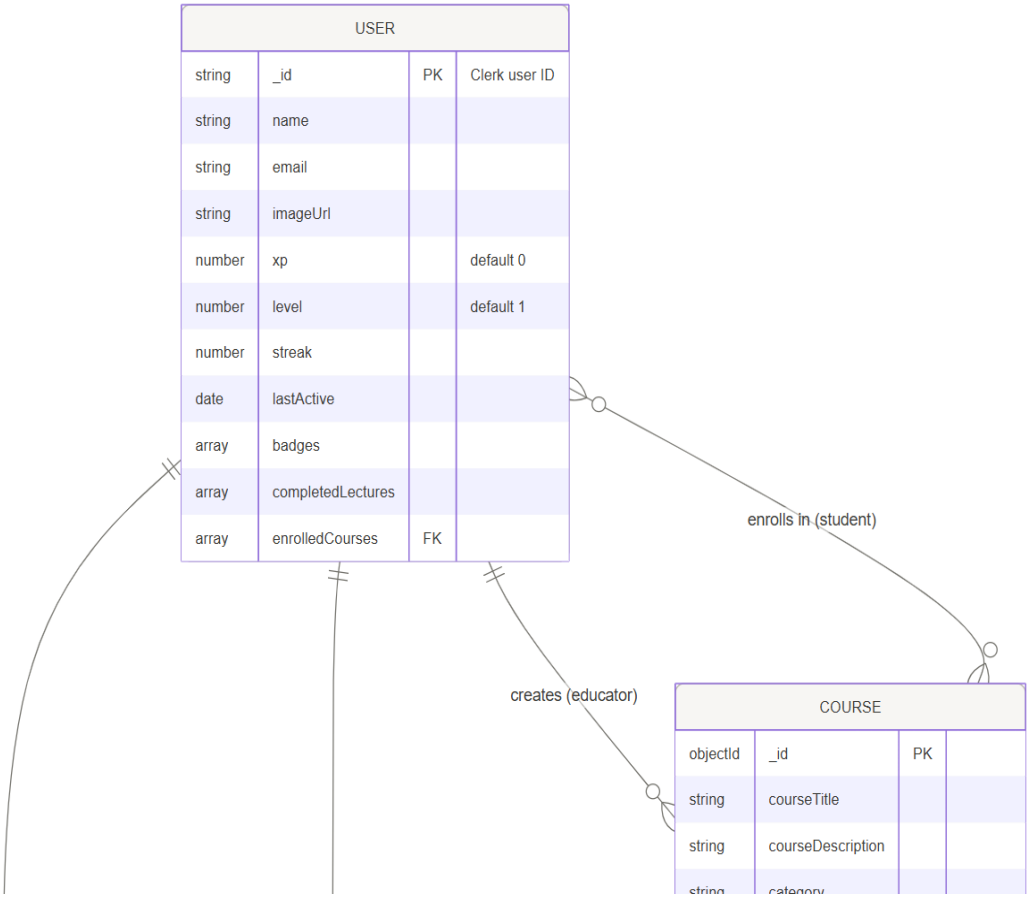
All data is stored in MongoDB, hosted on MongoDB Atlas. The server uses Mongoose to define schemas and interact with the database. There are four main collections: User, Course, CourseProgress, and Purchase. Course images and videos are uploaded by educators and stored as URLs inside the database.

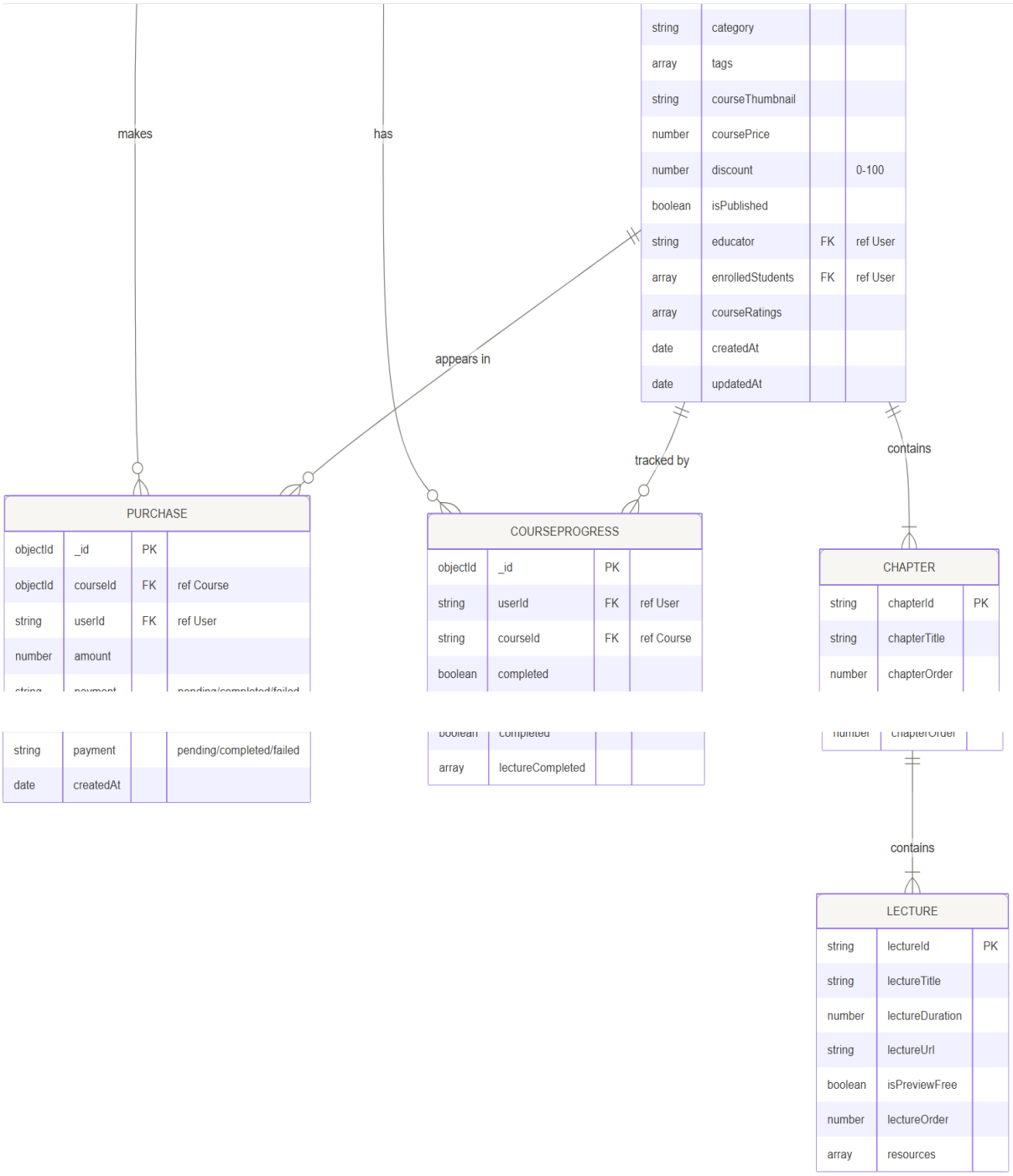
External Services

- Clerk — handles user sign-up, login, and session tokens. The backend verifies Clerk tokens on every protected request.
- Stripe — handles course payments. When a student purchases a course, Stripe processes the payment and sends a webhook to the backend to confirm it.

4.2 Database Design

ER Diagram:





ER Diagram 1

Schema:

User

User		
Field	**Type**	**Constraint / Notes**

PK _id	String	Clerk user ID (primary key)
name	String	Required
email	String	Required
imageUrl	String	Profile photo URL
FK enrolledCourses	[ObjectId]	References Course collection
xp	Number	Experience points, default 0
level	Number	Default 1
streak	Number	Consecutive active days
lastActive	Date	
badges	[String]	Earned badge IDs
completedLectures	[Object]	{lectureId, courseId, completedAt}
createdAt	Date	Auto — timestamps: true
updatedAt	Date	Auto — timestamps: true

User schema 1

Course

Course		
Field	**Type**	**Constraint / Notes**

PK _id	ObjectId	Auto-generated
courseTitle	String	Required
courseDescription	String	Required
category	String	
tags	[String]	
courseThumbnail	String	Image URL
coursePrice	Number	Required
discount	Number	0–100
isPublished	Boolean	Default true
FK educator	String	References User (Clerk ID)
FK enrolledStudents	[String]	References User (Clerk IDs)
courseContent	[Chapter]	Nested — see sub-schemas below
courseRatings	[Object]	{userId: String, rating: 1–5}
createdAt	Date	Auto — timestamps: true
updatedAt	Date	Auto — timestamps: true

Course Schema 1

CourseProgress

CourseProgress		
Field	**Type**	**Constraint / Notes**
PK _id	ObjectId	Auto-generated
FK userId	String	References User (Clerk ID)
FK courseId	String	References Course
completed	Boolean	Default false
lectureCompleted	[]	Array of completed lecture IDs

Course Progress Schema 1

Purchase

Purchase		
Field	**Type**	**Constraint / Notes**
PK _id	ObjectId	Auto-generated
FK courseId	ObjectId	References Course
FK userId	String	References User (Clerk ID)
amount	Number	Amount charged
payment	String	pending | completed | failed
createdAt	Date	Auto — timestamps: true

updatedAt	Date	Auto — timestamps: true

Purchase Schema 1

Nested Sub-schemas

The Course document contains two levels of embedded sub-schemas. This avoids separate database lookups when loading course content.

Chapter (embedded in Course)		
Field	**Type**	**Constraint / Notes**
chapterId	String	Required
chapterOrder	Number	Display order
chapterTitle	String	Required
chapterContent	[Lecture]	Array of Lecture sub-documents

Chapter Schema 1

Lecture (embedded in Chapter)		
Field	**Type**	**Constraint / Notes**
lectureId	String	Required
lectureTitle	String	Required
lectureDuration	Number	In minutes
lectureUrl	String	Video URL
isPreviewFree	Boolean	Whether non-enrolled users can view

lectureOrder	Number	Display order
resources	[Object]	{url, name} — downloadable files

Lecture Schema 1

4.3 User Interface Design

Student Portal — Pages and UX Decisions

Page / Screen	Key UI Elements	UX Decision & Rationale
Home	Hero banner with search bar, featured course grid, company logos strip, call-to-action section, recommended courses	Search is placed above the fold so users can find courses immediately. Company logos build trust before a student decides to enroll.
Course List	Search result count, active filter pill, 4-column course card grid, empty state message	The filter pill shows what is currently being searched and lets the user clear it in one click.
Course Details	Course thumbnail, title, star rating, price with discount badge, Enroll button, chapter accordion, free preview labels, instructor info	The accordion hides lecture details by default to keep the page readable. Free-preview labels let students sample content before paying.

My Enrollments	Grid of enrolled course cards, progress bar per card, Continue button	Progress bars show how far along each course is, motivating students to return.
Video Player	Video player, collapsible lecture sidebar, resource download links, Mark Complete button, star rating widget	Marking a lecture complete updates XP and streak instantly. The rating widget is placed right on the player page to make feedback easy.
Gamification Dashboard	XP progress bar, level badge, streak count, earned badges grid	Visible progress indicators motivate students to keep learning. Streaks encourage daily visits. Badges reward reaching key milestones.

Student Portal 1

Educator Portal — Pages and UX Decisions

Page / Screen	Key UI Elements	UX Decision & Rationale
Educator Dashboard	Total revenue card, total enrolment count, number of total courses	Key numbers are shown immediately on login so educators do not have to navigate to find their performance data.
Add Course	Form fields for title, description, category, price, discount, chapter	The course builder lets educators add chapters and lectures one at a time

	and lecture builder, publish button	without navigating away. The publish button means a course can be published.
My Courses	Grid of the educator's courses showing title, enrolment count and add course buttons	Enrolment counts are shown on the card so educators can see which courses are popular at a glance. Edit is one click from this page.
Course Management	Editable chapter and lecture list, price, discount, video, category, tag	Lectures can be reordered and edited inline. Resources are attached to individual lectures so students find them in context.
Student Enrollments	Table of students enrolled in a selected course: name, email, enrolment date	Helps educators to know who have enrolled in the courses

Educator Portal 1

Key Components (Reusable)

Component	Purpose
SearchBar	Text input that filters courses by title; used on the home page and course list
CourseCard	Displays a course thumbnail, title, rating, and price; used in grids across the student portal

Rating	Five-star click widget that sends a rating to the backend; used on the video player page
Navbar (Student)	Top navigation with nameand login/profile links for the student portal
Navbar (Educator)	Top navigation for the educator portal
Sidebar (Educator)	Left-side navigation linking to all educator pages
Footer	Site footer with informations; shared across student pages
Loading	Spinner shown while data is being fetched
Hero	Full-width banner on the home page with headline
CoursesSection	Fetches and displays a grid of featured courses on the home page
CallToAction	Prompts non-logged-in users to sign up; shown at the bottom of the home page
Companies	Displays logos of companies
Recommendations	Shows courses related to the previous enrollments

Key components 1

Chapter-5
Implementation

5.1 Development Environment

The Learnify LMS was developed using a modern web development environment.

Component	Tool Used
Code Editor	Visual Studio Code
Runtime	Node.js
Package Manager	npm
Version Control	Git & GitHub
API Testing	Postman

Development environment 1

5.2 Technologies Used (Modern Tools)

Frontend Technologies

Technology	Purpose	Why Selected
React.js	Build user interface	Fast, reusable components
Vite	React development tool	Faster build and hot reload
Tailwind CSS	Styling	Easy and responsive UI
React Router	Navigation	Smooth page routing
Axios	API communication	Simple HTTP requests
React YouTube	Video player	Easy YouTube integration
React Toastify	Notifications	Attractive alert messages

Frontend technologies 1

Backend Technologies

Technology	Purpose	Why Selected
Node.js	Server runtime	Fast and scalable
Express.js	Backend framework	Lightweight and easy routing

Clerk	Authentication	Secure user login and management
Stripe	Payment Gateway	Secure online payments

Backend technologies 1

Database

Technology	Purpose	Why Selected
MongoDB	Database	Flexible NoSQL database
Mongoose	Database Modeling	Simplifies MongoDB operations

Database 1

Why These Technologies Were Selected

- React.js was chosen for creating reusable and interactive UI components.
- Vite provides a much faster development experience than traditional React setups.
- Tailwind CSS allows responsive designs with less CSS code.
- Node.js and Express.js efficiently handle backend APIs.
- MongoDB stores flexible course and user data.
- Clerk provides secure authentication with minimal configuration.
- Stripe enables safe online course payments.

How Modern Tools Improved Development

- Faster development using Vite Hot Reload.
- Reusable React components reduced duplicate code.
- Tailwind CSS simplified responsive design.
- MongoDB allowed flexible data storage.
- Clerk reduced authentication implementation time.
- Git and GitHub made version management easier.

5.3 Module-wise Implementation

1. Authentication Module
   - User registration and login using Clerk.
   - Secure authentication and session management.
2. Course Management Module
   - Educators can create, edit, publish, and delete courses.
   - Add chapters, lectures, videos, and downloadable resources.

3. Student Learning Module
   - Students can browse, enroll, and watch course lectures.
   - Track lecture completion and course progress.
4. Payment Module
   - Stripe integration for secure course purchases.
   - Enrollment is automatically updated after successful payment.
5. Gamification Module
   - Awards XP after lecture completion.
   - Tracks levels, badges, streaks, and leaderboard rankings.
6. Rating Module
   - Students can rate completed courses.
   - Ratings are displayed on the course page.
7. Resource Management Module
   - Educators upload lecture resources.
   - Students download resources while watching lectures.

5.4 Code Structure & Key Snippets

Key Code Snippet 1: User Authentication

```
const {getToken}=useContext(AppContext)
```

Explanation:

Retrieves the authenticated user's token for secure API requests.

Key Code Snippet 2: Enroll Course

```
const token=await getToken()
const {data}=await axios.post(backendUrl+'/api/user/purchase',
  {courseId:courseData._id}, {headers:{Authorization:`Bearer ${token}`}})
```

Explanation:

Sends the enrollment request after successful payment.

Key Code Snippet 3: Watch Lecture

```
<YouTube
videoId={getYouTubeID(playerData.lectureUrl)}
opts={opts}
```

Explanation:

Loads and plays the selected YouTube lecture inside the application.

Key Code Snippet 4: Mark Lecture Complete

```
const {data}=await axios.post(backendUrl+'/api/user/update-course-progress',
  {courseId, lectureId}, {headers:{Authorization:`Bearer ${token}`}})
```

Explanation:

Updates the student's completed lectures and progress.

Key Code Snippet 5: XP Reward

```
if(data.success){
  toast.success("Lecture Completed! +50 XP earned")
```

Explanation:

Displays a success message and rewards XP after lecture completion.

Key Code Snippet 6: Download Resources

```
<a
  key={i}
  href={`${backendUrl}/api/resource/download/${filename}`}
  download={file.name}
  className='inline-flex items-center gap-2 bg-blue-50
   hover:bg-blue-100 border border-blue-200 text-blue-700
    dark:bg-gray-800 dark:text-gray-300 dark:border-gray-700
    px-3 py-1.5 rounded-lg text-sm font-medium transition-colors'
>
  {file.name}
</a>
```

Explanation:

Allows students to download lecture materials uploaded by the educator.

Chapter-6
Testing & Validation

6.1 Testing Strategy

The Learnify LMS was tested using manual testing to verify that every feature worked correctly. Each module was tested individually and then integrated to ensure the complete system functioned as expected. API requests were also tested using Postman.

Testing Approaches

- **Manual Testing:** Checked all features by interacting with the application.
- **Functional Testing:** Verified that every function produced the expected result.
- **Integration Testing:** Ensured frontend, backend, and database communicated correctly.
- **API Testing:** Tested backend APIs using Postman.

6.2 System Testing

The complete system was tested using the following test cases.

Test Case	Expected Result	Status
User Registration/Login	User can register and log in successfully	Pass
Browse Courses	Courses are displayed correctly	Pass
Course Search	Search returns matching courses	Pass
Course Enrollment	User is enrolled after successful payment	Pass
Watch Lecture	Video plays successfully	Pass
Watch Large Mode	Video opens in fullscreen mode	Pass
Mark Lecture Complete	Progress is updated correctly	Pass
Download Resources	Resource file downloads successfully	Pass
Submit Rating	Course rating is saved successfully	Pass
XP and Badge Update	XP, badges, and level update correctly	Pass
Leaderboard	User rankings are displayed correctly	Pass

6.3 User Acceptance Testing

The system was demonstrated to a small group of users who tested its main features and provided feedback.

Feedback Received

- The interface is clean and easy to use.
- Course navigation is simple.
- Watching lectures is smooth.
- Gamification features make learning more engaging.
- Resource downloading is useful for offline study.

Chapter-7
Project Management & Teamwork

7.1 Project Planning

The project was divided into smaller tasks to ensure systematic development. Each phase was completed before moving to the next, following a Work Breakdown Structure (WBS).

Work Breakdown Structure (WBS)

Phase	Activities
Requirement Analysis	Identify project goals and user requirements
Research	Study existing LMS platforms and technologies
System Design	Design database, UI, and system architecture
Frontend Development	Develop student and educator interfaces
Backend Development	Create APIs, authentication, and database integration
Feature Implementation	Add payment, gamification, and resources
Testing	Perform manual testing and fix bugs
Documentation	Prepare report and presentation

WBS 1

7.2 Individual & Team Work

Since this was an individual project, all stages of development were completed independently.

Responsibilities

Task	Responsibility
Requirement Analysis	Self
System Design	Self
Frontend Development	Self
Backend Development	Self

Database Design	Self
API Development	Self
Testing	Self
Documentation	Self

Responsibility 1

Development Process

The project was completed by planning, designing, implementing, testing, and documenting each module one after another. This approach ensured that every feature was fully functional before adding the next one.

Development Tools

Tool	Purpose
Git	Version control
GitHub	Source code backup
Visual Studio Code	Code editor
Postman	API testing
MongoDB Compass	Database management

Development tools 1

Challenges Faced and Solutions

Challenge	Solution
YouTube video playback issue	Implemented a function to extract the correct YouTube video ID.
Resource upload and download	Integrated Cloudinary for storing and retrieving files.
Gamification updates	Updated backend logic to correctly award XP, badges, and levels.
Payment integration	Configured and tested Stripe using test mode.
Responsive UI	Used Tailwind CSS responsive classes for different screen sizes.

Challenges 1

7.3 Timeline

Project Schedule

Phase	Duration
Requirement Analysis	Month 1 (Week 1–2)
Research & Literature Review	Month 1 (Week 3–4)
System Design (Architecture, Database, UI)	Month 2
Frontend Development	Month 3
Backend Development	Month 4 (Week 1–2)
Feature Integration & API Testing	Month 4 (Week 3–4)
System Testing & Debugging	Month 5 (Week 1–2)
Documentation & Presentation Preparation	Month 5 (Week 3–4)

Task	Month 1	Month 2	Month 3	Month 4	Month 5
Requirement Analysis	●				
Research & Literature Review	●				
System Design		●			
Frontend Development			●		
Backend Development				●	
Feature Integration				●	
Testing & Debugging					●
Documentation & Presentation					●

Gantt Chart 1

7.4 Budget & Cost Analysis

Most of the software and services used in this academic project were available under free plans.

Item	Estimated Cost
Visual Studio Code	Free
React.js	Free

Node.js	Free
MongoDB Community Edition	Free
Git & GitHub	Free
Clerk (Free Tier)	Free
Stripe Test Environment	Free
Hosting (Development)	Free
Total Estimated Cost	**$0**

Budget and Cost 1

Cost Analysis

The project was developed entirely using free and open-source tools, making it economical while still providing all the required features of a modern Learning Management System.

7.5 Risk Management

Potential Risk	**Impact**	**Mitigation Strategy**
Authentication failure	High	Used Clerk authentication with secure token validation.
Payment errors	High	Tested the payment flow using Stripe's sandbox environment before deployment.
Data loss	High	Maintained source code on GitHub and backed up the MongoDB database regularly.
Video playback issues	Medium	Validated YouTube URLs and extracted

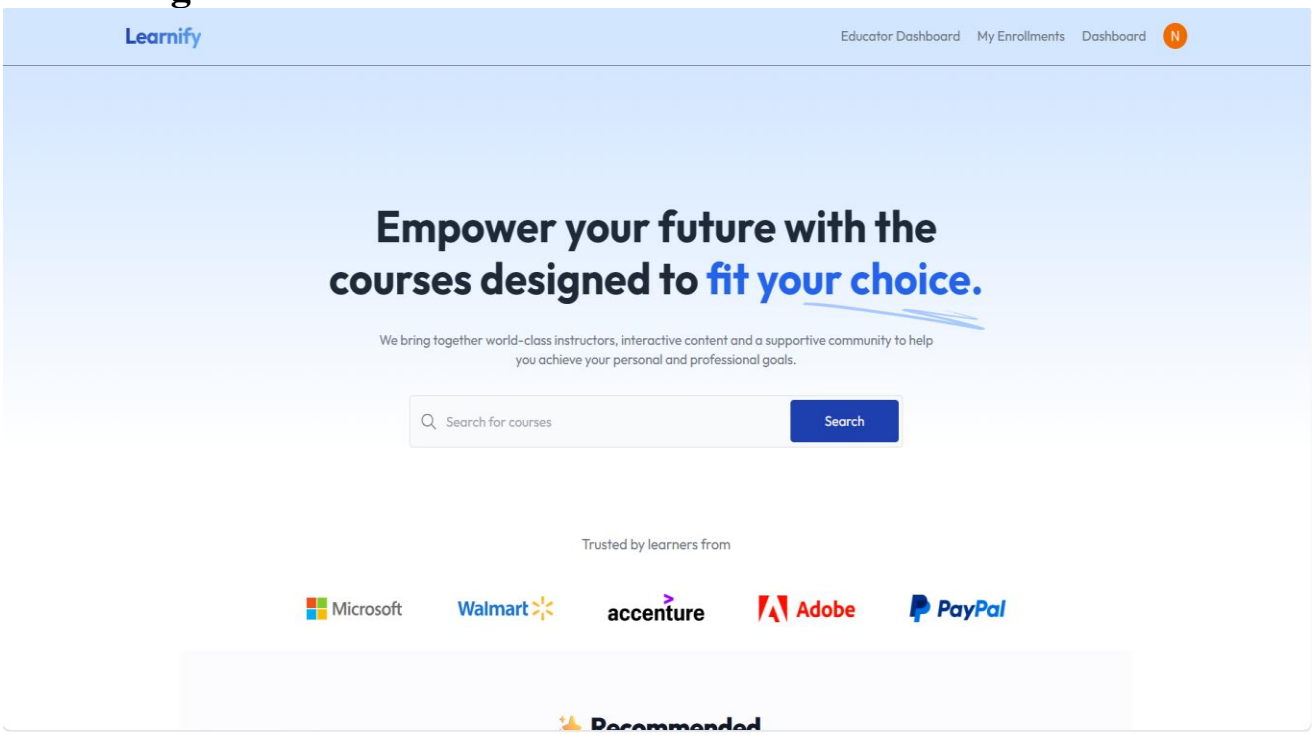
		the correct video ID before loading videos.
Resource upload failure	Medium	Verified file uploads.

Potential Risk 1

Chapter-8
Results & Discussion

8.1 System Output

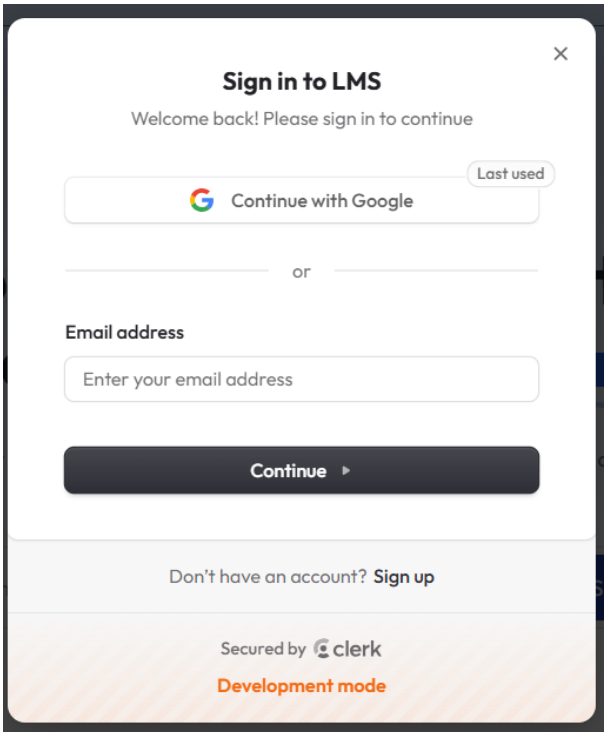
Home Page:



Home Page 1

Displays featured courses and navigation options.

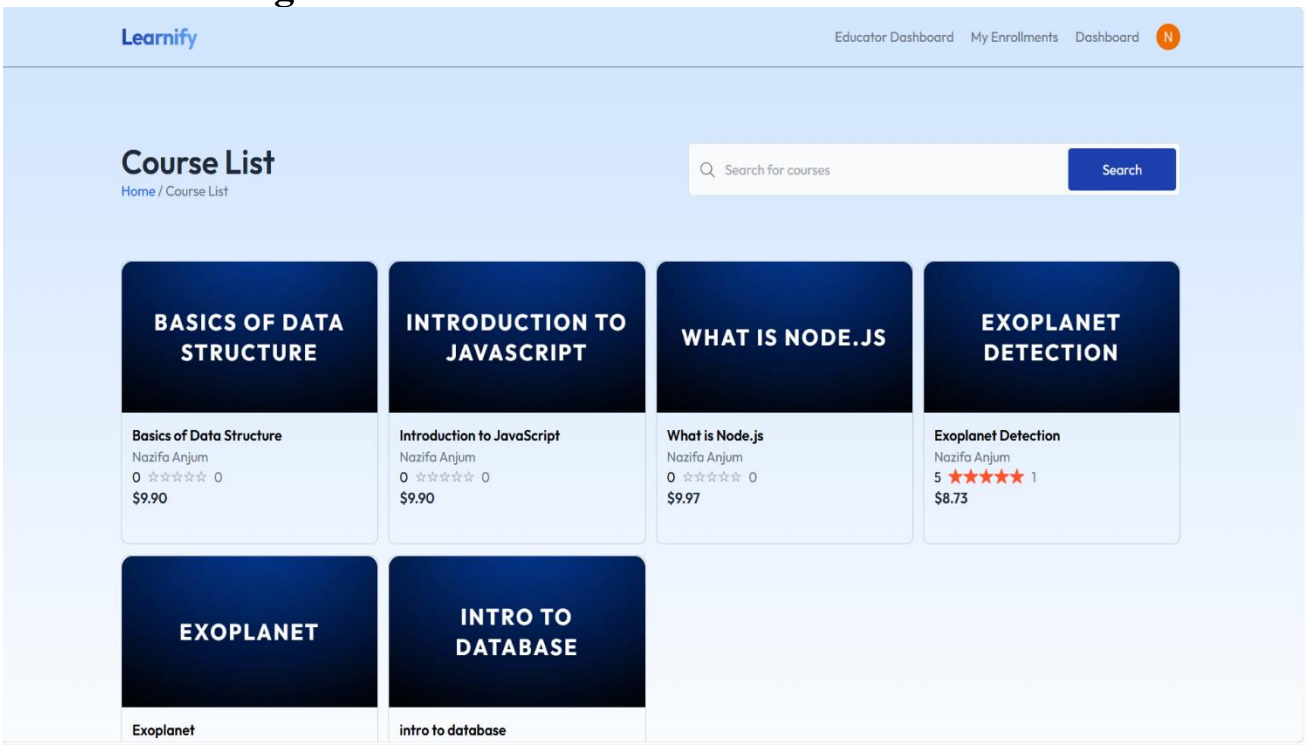
Login Page:



Login Page 1

Secure authentication using Clerk.

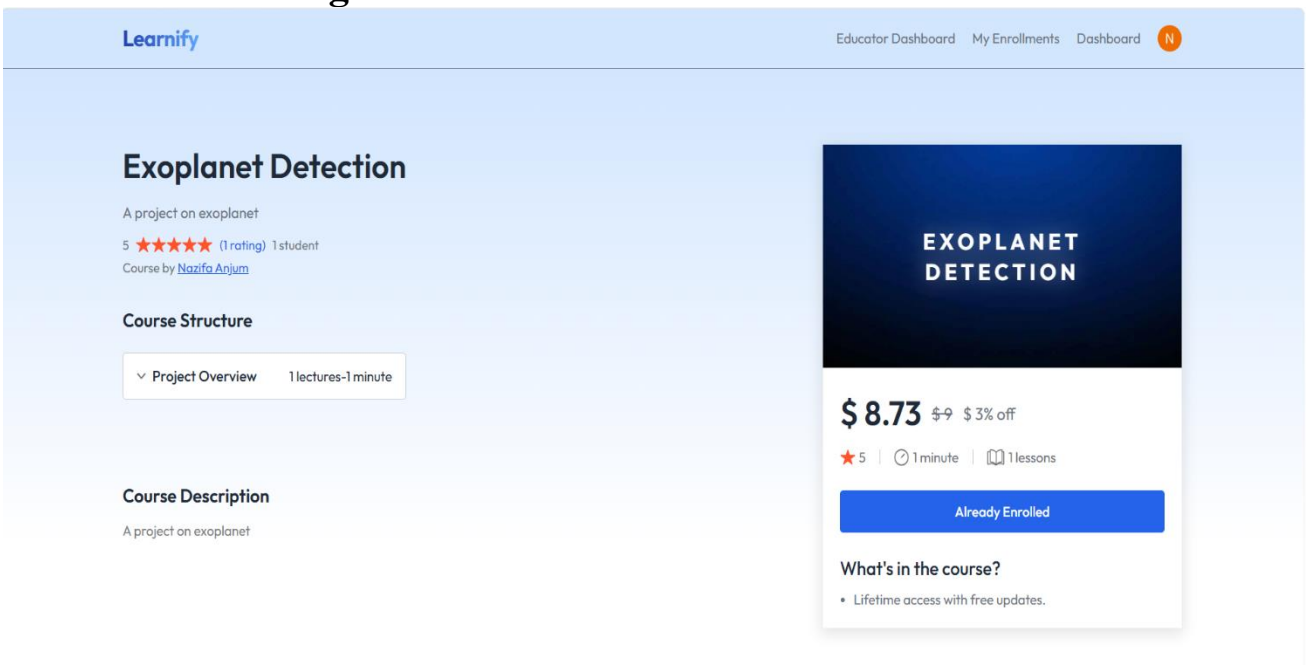
Course List Page:



Course List Page 1

Shows all the courses.

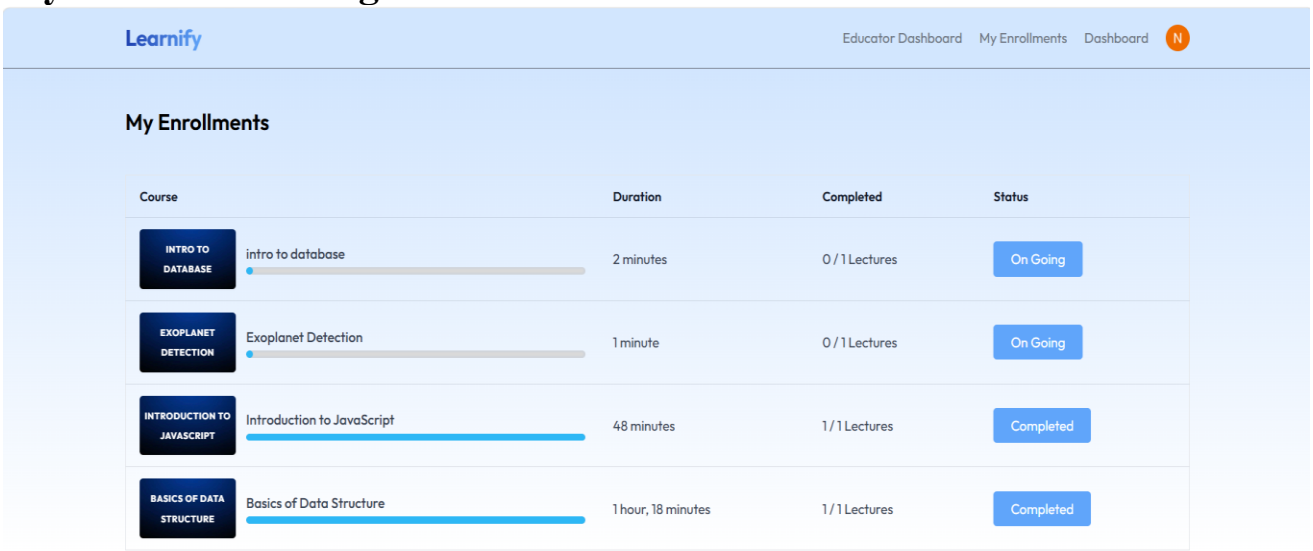
Course Details Page:



Course Details Page 1

Shows details of courses.

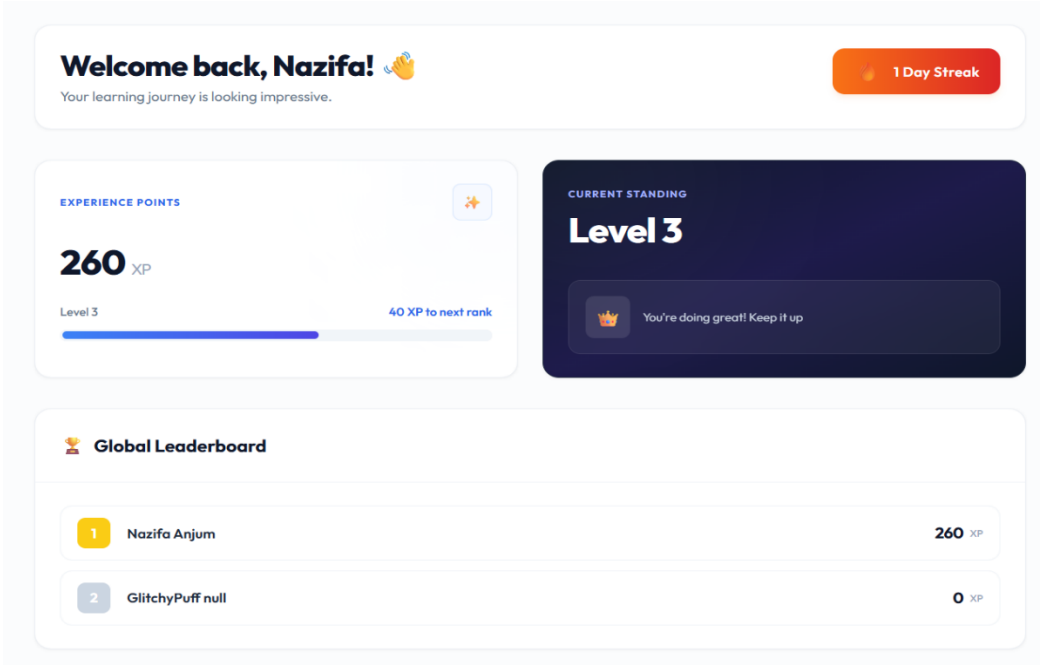
My Enrollments Page:



My Enrollments Page 1

Shows all the enrolled courses and progress of those courses.

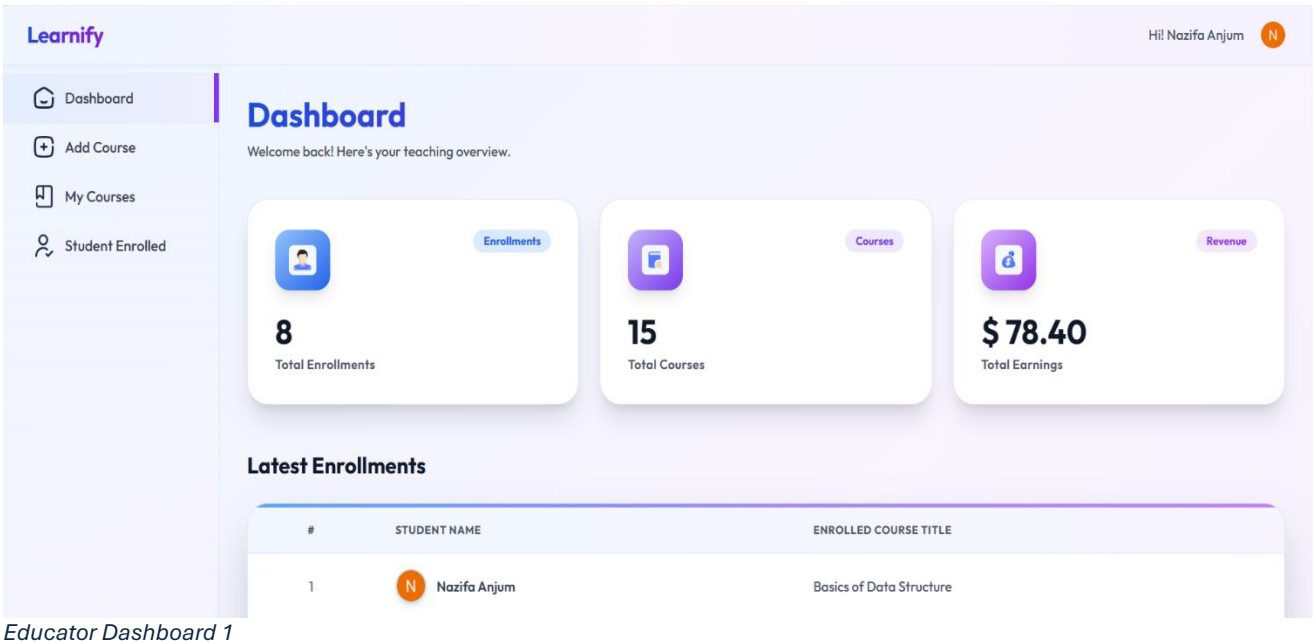
Dashboard:



Dashboard 1

Shows xp, levels, streak, leaderboard.

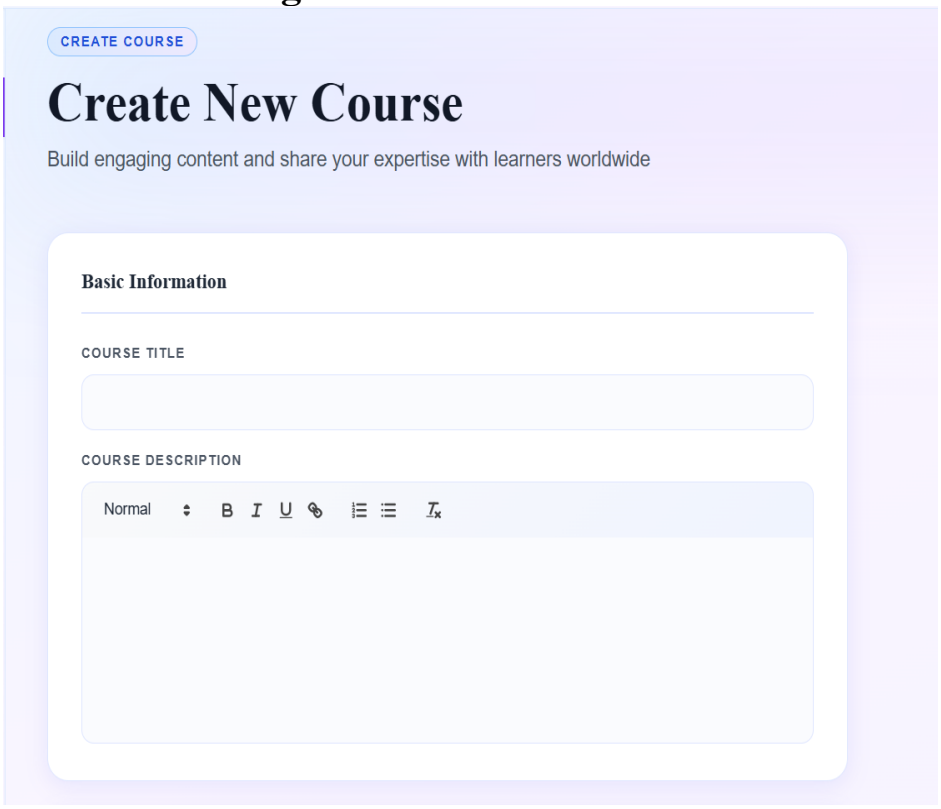
Educator Dashboard:



Educator Dashboard 1

Shows total enrollments, total courses, total earnings, latest enrollments.

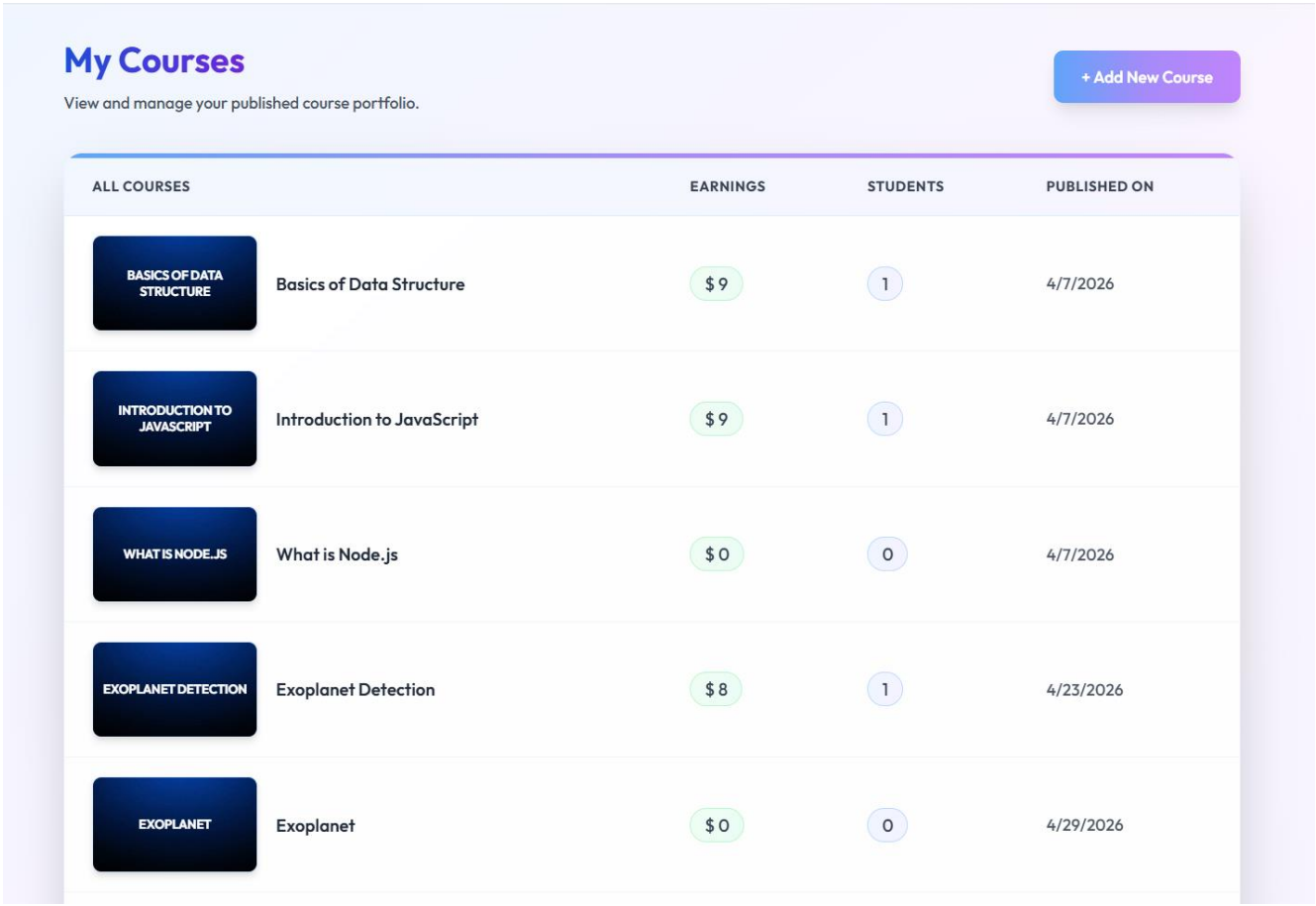
Add Course Page:



Add Course Page 1

Educators can create courses by adding chapters, lectures, prices, discount, tags, categories, videos.

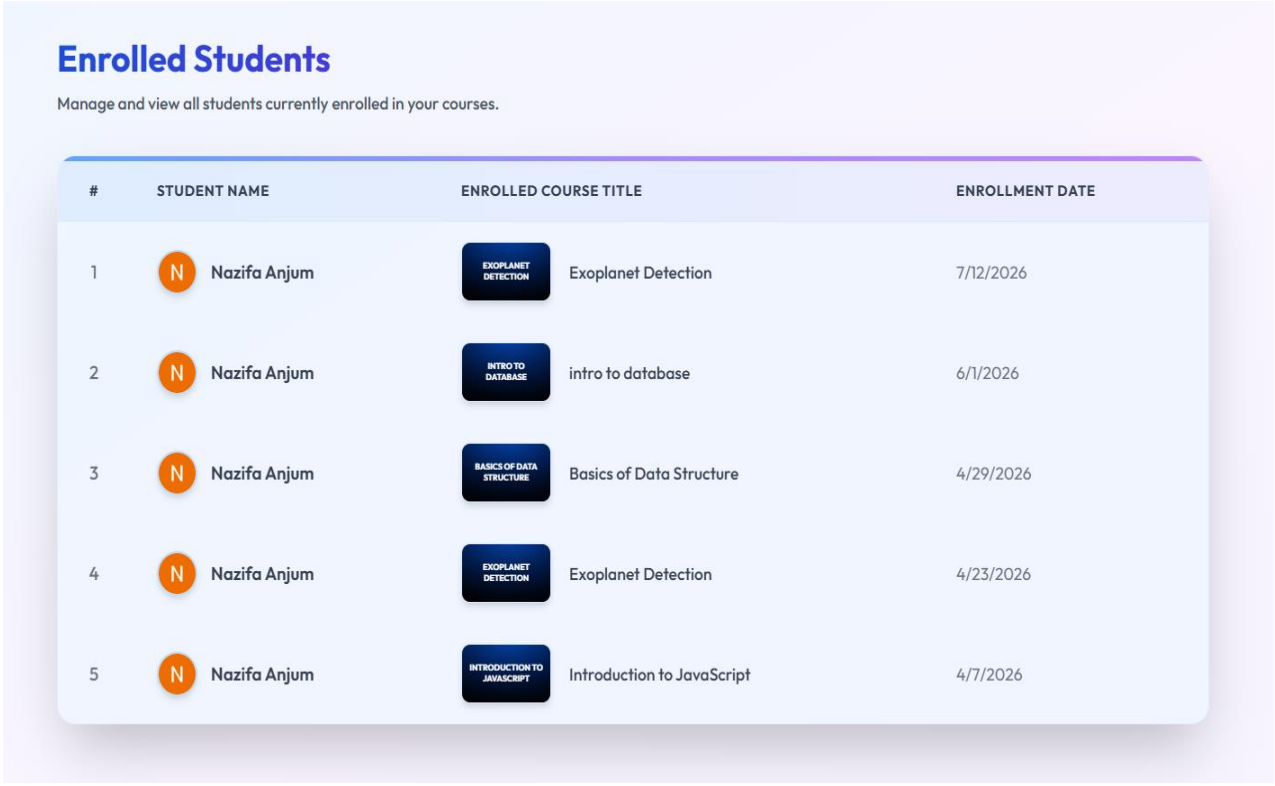
My Courses Page:



My Courses Page 1

Shows all the courses of the educator.

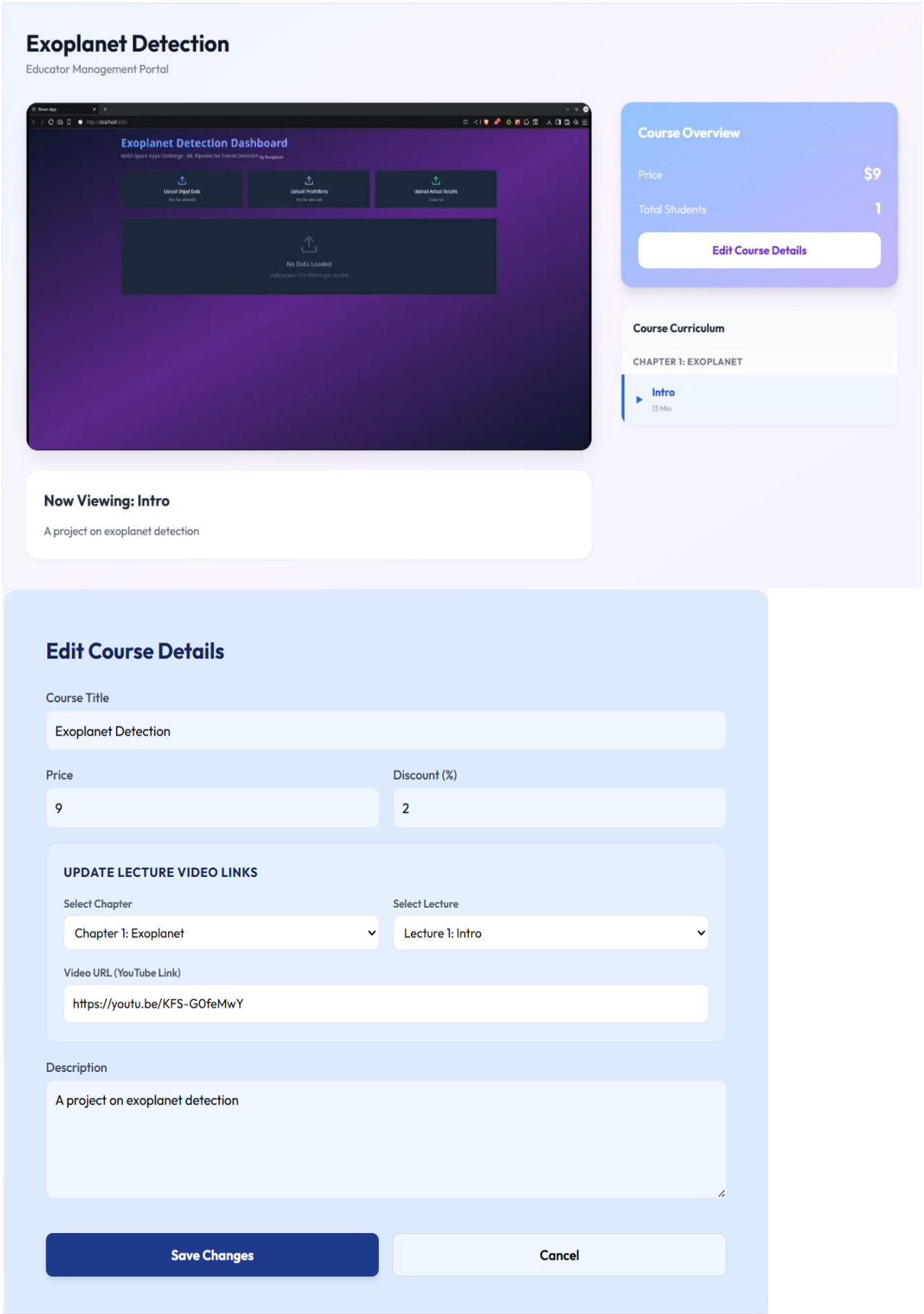
Students Enrollments Page:



Students Enrolled Page 1

Shows all the students enrolled in the courses.

Course Management Page:



Course Management Page 1

Allows courses to be edited or updated.

8.2 Performance Evaluation

The system was evaluated based on responsiveness, functionality, and user experience.

Performance Metrics

Metric	Result
Home Page Load Time	~1.8 seconds
Course Details Load Time	~1.5 seconds
Lecture Video Loading	~2.0 seconds
API Response Time	150–300 ms
Authentication Time	~1 second
Database Query Time	<200 ms
Resource Download	Depends on file size and internet speed

8.3 Comparison with Existing Systems

Feature	Udemy	Coursera	Moodle	Google Classroom	Khan Academy	Learnify (Proposed)
Video Lectures	✓	✓	✓	✓	✓	✓
Course Enrollment	✓	✓	✓	✓	✗	✓
Online Payment	✓	✓	✗	✗	✗	✓
Downloadable Resources	✓	✓	✓	✓	✓	✓
Progress Tracking	✓	✓	✓	Basic	✓	✓
Course Ratings	✓	✓	Limited	✗	✗	✓
Gamification (XP, Badges, Leaderboard)	Limited	✗	Plugin	✗	Basic	✓
Educator Dashboard	✓	✓	✓	✓	✗	✓
Student Dashboard	✓	✓	✓	Basic	✓	✓

Improvements and Advantages

Compared with existing Learning Management Systems, Learnify offers several improvements:

- Combines course management, online payment, and gamification in a single platform.
- Provides XP, badges, levels, and leaderboards to increase student motivation.
- Allows educators to upload lecture resources for students to download.
- Offers an intuitive and responsive user interface for both students and educators.
- Includes secure authentication using Clerk and online payments through Stripe.
- Supports lecture completion tracking, course ratings, and progress monitoring to enhance the overall learning experience.

Overall, the project successfully achieved its objectives by delivering a modern, user-friendly Learning Management System with features that improve student engagement and simplify course management.

Chapter-9
Conclusion & Future Work

9.1 Conclusion

The Learnify Learning Management System was successfully designed and implemented as a modern web-based platform for online learning. The system provides essential features such as course management, secure authentication, online payments, progress tracking, downloadable resources, and gamification. These features fulfilled the project objectives by creating an user-friendly learning environment for both students and educators.

9.2 Limitations

Although the system is functional, it has some limitations:

- The platform currently supports only YouTube videos for course lectures.
- Students cannot download lecture videos for offline viewing.
- There is no discussion forum or chat system for communication between students and educators.
- Email and push notifications are not implemented for course updates or announcements.
- The system does not support certificate generation after course completion.

9.3 Future Enhancements

The system can be further improved by adding the following features:

- Built-in live video conferencing.
- Email and push notification support for announcements and reminders.
- Advanced analytics dashboard for educators and administrators.
- Discussion forums and peer-to-peer collaboration features.
- Multi-language support and improved accessibility features.

REFERENCES

[1] React Team, *React Documentation*. [Online]. Available: https://react.dev/. [Accessed: 13-Jul-2026].

[2] Tailwind CSS, *Tailwind CSS Documentation*. [Online]. Available: https://tailwindcss.com/docs. [Accessed: 13-Jul-2026].

[3] Node.js Foundation, *Node.js Documentation*. [Online]. Available: https://nodejs.org/docs. [Accessed: 13-Jul-2026].

[4] Express.js, *Express.js Documentation*. [Online]. Available: https://expressjs.com/. [Accessed: 13-Jul-2026].

[5] MongoDB Inc., *MongoDB Documentation*. [Online]. Available: https://www.mongodb.com/docs/. [Accessed: 13-Jul-2026].

[6] Mongoose, *Mongoose Documentation*. [Online]. Available: https://mongoosejs.com/docs/. [Accessed: 13-Jul-2026].

[7] Clerk, *Clerk Documentation*. [Online]. Available: https://clerk.com/docs. [Accessed: 13-Jul-2026].

[8] Stripe, *Stripe API Documentation*. [Online]. Available: https://stripe.com/docs. [Accessed: 13-Jul-2026].

[09] Google Developers, *YouTube IFrame Player API Reference*. [Online]. Available: https://developers.google.com/youtube/iframe_api_reference. [Accessed: 13-Jul-2026].

[10] GitHub, *GitHub Documentation*. [Online]. Available: https://docs.github.com/. [Accessed: 13-Jul-2026].

APPENDICES

Appendix A: API Routing Reference

These RESTful endpoints map directly to your backend collections (User, Course, Purchase, and CourseProgress).

1. Course Management (/api/courses)

- GET / – Fetch all published courses with pagination and category filtering.
- POST /create – Create a new course skeleton (Requires Educator role).
- GET /:id – Retrieve a full course layout, including its chapters, lectures, and resources.
- PUT /:id/chapters – Append or reorder sections matching chapterOrder.

2. Learning Progress & Gamification (/api/progress)

- GET /:courseId – Fetch active user's tracking history from CourseProgress.
- POST /update-lecture – Marks a lectureId as completed, appends it to lectureCompleted, pushes to User.completedLectures, updates streak, and calculates current xp / level.

3. Commercial Checkout (/api/payments)

- POST /checkout – Creates a payment intent and instantiates a Purchase record with a pending status.
- POST /webhook – Asynchronous listener that catches your gateway's validation payload, transitions Purchase.payment to completed, and appends the courseId to User.enrolledCourses.

Appendix B: Environmental Setup:

Environmental setup to run the project:

Project Initialization:

i. Create a main project folder named LMS and open it in VS Code.

ii. Inside the project, create a folder named client for the frontend.

iii. Open the terminal and create a React project using Vite inside the client folder.

iv. After the project is created, install all required packages by running npm install.

Frontend Configuration:

1. Install Tailwind CSS in the client folder to design the user interface.
2. Add Google Fonts to improve the text style of the website.
3. Install Clerk Authentication to handle user login and registration.
4. Create a .env file inside the client folder and add the Clerk publishable key.
5. Wrap the main application with Clerk's provider so authentication works throughout the app.

Backend Setup:

1. Create a folder named server for the backend and install required packages using npm install.
2. Create a .env file inside the server folder and define the server port number.
3. Connect the backend to MongoDB Atlas by adding the database URL to the .env file and writing a connection function.
4. Add Stripe publishable and secret keys to the .env file to enable online payments.

Deployment:

1. Upload the entire project to GitHub.
2. Deploy the backend (server folder) to Vercel and add all server environment variables.
3. Deploy the frontend (client folder) to Vercel and add all frontend environment variables.